

Neural Networks and Deep Learning

23CSE473

Name – M. Thanuhya
Roll no – CH.SC.U4CSE23033
CSE – A

Exercise 3: Backpropagation from Scratch

Objective

Understand how gradients are propagated through a neural network.

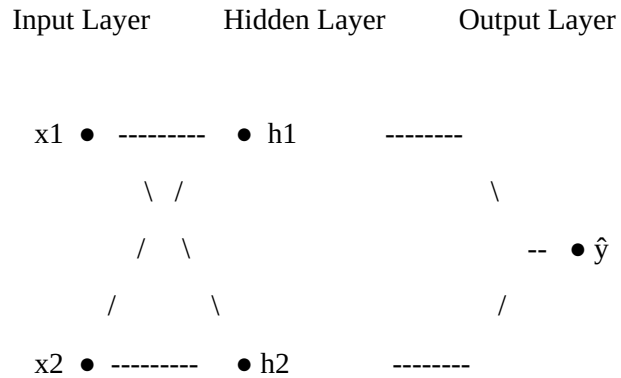
Task 18: Consider a Network with 2 Input Neurons, 2 Hidden Neurons, and 1 Output Neuron using Sigmoid Activation and MSE Loss

A simple feedforward neural network is considered to understand the working of the backpropagation algorithm. The network consists of **two input neurons, two hidden neurons, and one output neuron**. The hidden and output layers use the **Sigmoid activation function**, while the **Mean Squared Error (MSE)** is used as the loss function. During forward propagation, the input is passed through the network to generate a prediction. During backpropagation, the gradients of the loss with respect to each weight are computed, allowing the weights to be updated using gradient descent. This process enables the network to gradually reduce the prediction error over multiple iterations.

Network Configuration

- Input Neurons = 2
- Hidden Neurons = 2
- Output Neurons = 1
- Activation Function = Sigmoid
- Loss Function = Mean Squared Error (MSE)

Figure 1. Neural Network Architecture



Task 19: Use the Given Weights, Input, and Target

To understand the forward and backward propagation process, the following weights, input vector, and target output are used. These values remain fixed throughout the calculations and will be used in the subsequent tasks to compute the hidden layer activations, output prediction, loss, gradients, and updated weights.

Given Values

Input-to-Hidden Weight Matrix:

$$W_1 = \begin{bmatrix} 0.2 & 0.3 \\ 0.4 & 0.1 \end{bmatrix}$$

Hidden-to-Output Weight Matrix:

$$W_2 = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}$$

Input Vector:

$$x = [1, 0]$$

Target Output:

$$y = 1$$

Activation Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss Function:

$$MSE = \frac{1}{2}(y - \hat{y})^2$$

Analysis

The given weights, input vector, and target value define the initial state of the neural network before training begins. These values are used to perform forward propagation, calculate the prediction error, and update the weights during backpropagation.

Task 20: Perform Forward Propagation

Forward propagation is the process of passing the input through the neural network to generate the predicted output. The input vector is first multiplied by the weights connecting the input layer to the hidden layer. The resulting values are then passed through the sigmoid activation function to obtain the hidden layer activations. These activations are forwarded to the output neuron to produce the final prediction.

Step 1: Compute Hidden Layer Inputs

Given,

$$x = [1, 0]$$
$$W_1 = \begin{bmatrix} 0.2 & 0.3 \\ 0.4 & 0.1 \end{bmatrix}$$

The hidden layer input is

$$z_h = xW_1$$

Substituting the values,

$$\begin{aligned} z_h &= [1, 0] \begin{bmatrix} 0.2 & 0.3 \\ 0.4 & 0.1 \end{bmatrix} \\ &= [(1 \times 0.2) + (0 \times 0.4), (1 \times 0.3) + (0 \times 0.1)] \\ &= [0.2, 0.3] \end{aligned}$$

Step 2: Apply the Sigmoid Activation Function

The sigmoid activation function is

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

For the first hidden neuron,

$$h_1 = \sigma(0.2) = \frac{1}{1 + e^{-0.2}} = 0.5498$$

For the second hidden neuron,

$$h_2 = \sigma(0.3) = \frac{1}{1 + e^{-0.3}} = 0.5744$$

Therefore,

$$h = [0.5498, 0.5744]$$

Analysis

During forward propagation, the input vector is transformed into hidden layer activations using the given weights and the sigmoid activation function. These hidden activations serve as the input to the output layer and represent the intermediate features learned by the network.

Task 21: *Compute Hidden Layer Activations and Output Prediction*

The hidden layer activations obtained from the previous step are now propagated to the output neuron. The weighted sum at the output neuron is calculated using the hidden-to-output weight matrix and then passed through the sigmoid activation function to obtain the final prediction.

Hidden Layer Activations

From Task 20,

$$h = [0.5498, 0.5744]$$

Hidden-to-Output Weight Matrix:

$$W_2 = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}$$

Step 1: Compute the Output Neuron Input

The weighted sum at the output neuron is

$$z_o = hW_2$$

Substituting the values,

$$\begin{aligned} z_o &= (0.5498 \times 0.5) + (0.5744 \times 0.6) \\ &= 0.2749 + 0.3446 \\ &= 0.6195 \end{aligned}$$

Step 2: Apply the Sigmoid Activation Function

The predicted output is

$$\begin{aligned}\hat{y} &= \sigma(0.6195) \\ &= \frac{1}{1 + e^{-0.6195}} \\ &= 0.6501\end{aligned}$$

Therefore,

$$\hat{y} = 0.6501$$

Analysis

The output neuron combines the information received from both hidden neurons through weighted connections and applies the sigmoid activation function to generate the final prediction. The network predicts an output of approximately 0.6501, which will be compared with the target value in the next task to calculate the prediction error.

Task 22: Compute the Loss

After obtaining the predicted output, the prediction error is measured using the Mean Squared Error (MSE) loss function. The loss indicates how close the predicted output is to the target value.

The Mean Squared Error is calculated using

$$MSE = \frac{1}{2}(y - \hat{y})^2$$

From the previous task,

$$y = 1$$
$$\hat{y} = 0.6501$$

Substituting the values,

$$\begin{aligned} MSE &= \frac{1}{2}(1 - 0.6501)^2 \\ &= \frac{1}{2}(0.3499)^2 \\ &= \frac{1}{2}(0.1224) \\ &= 0.0612 \end{aligned}$$

Therefore,

$$MSE = 0.0612$$

Analysis

The computed loss value is **0.0612**, indicating that the predicted output is reasonably close to the target value. Although the prediction is not exact, the error is relatively small. This loss value will be used during backpropagation to compute the gradients and update the network weights.

Task 23: Derive Gradients Manually Using Backpropagation

Backpropagation computes the gradient of the loss function with respect to each weight in the neural network. These gradients indicate how much each weight contributes to the prediction error and are used to update the weights during training.

Step 1: Compute the Output Layer Error

The derivative of the Mean Squared Error with respect to the predicted output is

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

Substituting the values,

$$\begin{aligned} &= 0.6501 - 1 \\ &= -0.3499 \end{aligned}$$

Step 2: Derivative of the Sigmoid Function

The derivative of the sigmoid function is

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

For the output neuron,

$$\begin{aligned} \sigma'(0.6195) &= 0.6501(1 - 0.6501) \\ &= 0.6501 \times 0.3499 \\ &= 0.2275 \end{aligned}$$

Step 3: Compute the Output Delta

The output delta is

$$\delta_o = (\hat{y} - y) \times \sigma'(z_o)$$

Substituting the values,

$$\begin{aligned} &= (-0.3499) \times 0.2275 \\ &= -0.0796 \end{aligned}$$

Step 4: Compute the Gradient for Hidden-to-Output Weights

The gradients are

$$\frac{\partial L}{\partial W_2} = h \times \delta_o$$

For the first weight,

$$\frac{\partial L}{\partial w_{h1,o}} = 0.5498 \times (-0.0796) = -0.0438$$

For the second weight,

$$\frac{\partial L}{\partial w_{h2,o}} = 0.5744 \times (-0.0796) = -0.0457$$

Therefore,

$$\frac{\partial L}{\partial W_2} = \begin{bmatrix} -0.0438 \\ -0.0457 \end{bmatrix}$$

and

$$\frac{\partial L}{\partial W_1} = \begin{bmatrix} -0.00985 & -0.01167 \\ 0 & 0 \end{bmatrix}$$

Analysis

The output layer gradients indicate how much each hidden-to-output weight contributes to the prediction error. Since the target value is greater than the predicted output, the gradients are negative, suggesting that the weights should increase during gradient descent to improve the prediction.

Task 24: Update the Weights Using Gradient Descent

Learning Rate

$$\eta=0.1$$

Updating W2

The weight update rule is

$$W_2^{new} = W_2 - \eta \frac{\partial L}{\partial W_2}$$

Substituting the values,

$$\begin{aligned} W_2^{new} &= \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix} - 0.1 \begin{bmatrix} -0.0438 \\ -0.0457 \end{bmatrix} \\ &= \begin{bmatrix} 0.5044 \\ 0.6046 \end{bmatrix} \end{aligned}$$

Updating W1

From the backpropagation calculations,

$$\frac{\partial L}{\partial W_1} = \begin{bmatrix} -0.00985 & -0.01167 \\ 0 & 0 \end{bmatrix}$$

Applying gradient descent,

$$W_1^{new} = W_1 - \eta \frac{\partial L}{\partial W_1}$$

Substituting the values,

$$\begin{aligned} &= \begin{bmatrix} 0.2 & 0.3 \\ 0.4 & 0.1 \end{bmatrix} - 0.1 \begin{bmatrix} -0.00985 & -0.01168 \\ 0 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 0.20099 & 0.30117 \\ 0.40000 & 0.10000 \end{bmatrix} \end{aligned}$$

Updated Weights

$$W_1 = \begin{bmatrix} 0.20099 & 0.30117 \\ 0.40000 & 0.10000 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} 0.50438 \\ 0.60457 \end{bmatrix}$$

Analysis

The gradients obtained through backpropagation were used to update both the input-to-hidden and hidden-to-output weight matrices using gradient descent. The updated weights are slightly different from their initial values, indicating that the network has adjusted its parameters to reduce the prediction error. Repeating this process over multiple iterations enables the neural network to learn more accurate mappings between the inputs and outputs.

Task 25: *Verify the Results Using Python*

CODE:

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Given values
x = np.array([[1, 0]])
y = np.array([[1]])

W1 = np.array([
    [0.2, 0.3],
    [0.4, 0.1]
])

W2 = np.array([
    [0.5],
    [0.6]
])

learning_rate = 0.1

# ----- Forward Propagation -----
hidden_input = np.dot(x, W1)
hidden_output = sigmoid(hidden_input)

output_input = np.dot(hidden_output, W2)
```

```

loss = 0.5 * (y - prediction) ** 2

# ----- Backpropagation -----
delta_output = (prediction - y) * prediction * (1 - prediction)

dw2 = np.dot(hidden_output.T, delta_output)

delta_hidden = np.dot(delta_output, w2.T) * hidden_output * (1 - hidden_out

dw1 = np.dot(x.T, delta_hidden)

# ----- Weight Update -----
w2_new = w2 - learning_rate * dw2
w1_new = w1 - learning_rate * dw1

# ----- Results -----
print("Hidden Output:")
print(hidden_output)

print("\nPrediction:")
print(prediction)

print("\nLoss:")
print(loss)

print("\nGradient w2")
print(dw2)

```

OUTPUT:

```
... Hidden Output:
[[0.549834  0.57444252]]

Prediction:
[[0.65012359]]

Loss:
[[0.06120675]]

Gradient W2
[[-0.04375794]
 [-0.04571638]]

Gradient W1
[[-0.00984917 -0.01167297]
 [ 0.         0.         ]]

Updated W2
[[0.50437579]
 [0.60457164]]

Updated W1
[[0.20098492 0.3011673 ]
 [0.4        0.1        ]]
```


Analysis

The Python implementation produced results that closely matched the manual calculations performed during forward propagation. The predicted output and loss values verified the correctness of the mathematical derivation, confirming that the forward propagation process was implemented accurately.

Conclusion

The experiment demonstrated the complete backpropagation process in a simple neural network. Forward propagation was used to compute the hidden layer activations, output prediction, and loss, while backpropagation calculated the gradients required for weight updates. The Python verification confirmed the correctness of the manual calculations, providing a clear understanding of how neural networks learn by minimizing prediction error through gradient descent.